Professional Training in Cybersecurity I

Module 1 — Session 3

Scapy and Raw Packet Crafting

Mohamed Gebril

Session Objective

By the end you can do each of these with Scapy, on packets you built yourself:

Build — a packet as layer objects
fields, defaults and computed fields, one class per header, in an interactive session

Stack — layers into a frame
the `/` operator, the binding between headers, and values outside the specification

Dissect — bytes back into fields
`.show()`, indexing by layer, the round trip to bytes, and the `Raw` remainder

Send — at layer 3 or layer 2
`send` and `sendp`, Scapy's own route and MAC lookup, and the privilege both need

Probe — and match the replies
`sr`, `sr1` and `srp`, answered and unanswered, timeouts and retries

Capture — live traffic and pcap files
`sniff()` with a filter and a callback, `AsyncSniffer`, `wrpcap` and `rdpcap`

Session Agenda

Part 1 — Scapy: Packets as Objects
the build by hand against one object per layer, fields, defaults and computed fields, one class per header

Part 2 — Crafting: Stacking Layers with `/`
binding, the `/` operator, one pattern across the taught headers, a value set over the binding, values outside the specification

Part 3 — Dissecting: From Bytes to Fields
bindings read in reverse, `.summary()` and `.show()`, indexing, the round trip, the `Raw` remainder

Part 4 — Sending: Packets onto the Wire
`send` and `sendp`, Scapy's own route and MAC lookup, the privilege boundary

Part 5 — Send and Receive: `sr`, `sr1`, `srp`
matching a reply to its request, answered and unanswered, layer 2, timeouts and retries

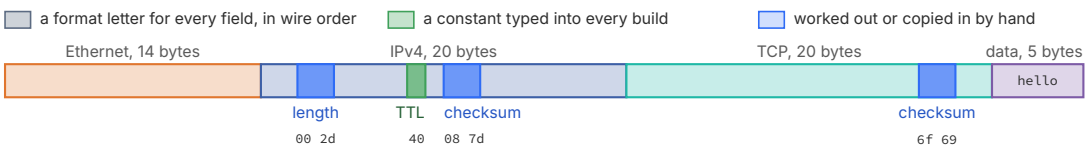
Part 6 — Capturing: `sniff()` and pcap
interface, filter, action and storage, `prn` and `AsyncSniffer`, pcap files, three tools for one goal

Scapy: Packets as Objects

The Problem: Every Header Field Set by Hand

by hand: one `pack` per header, inside out

```
tcp = struct.pack("!HHLLBBHHH", 50000, 80, SEQ, ACK, data_offset, 0x18, 0xFFFF, 0x6F69, 0)
total_length = 20 + len(tcp + data)
ip = struct.pack("!BBHHHBBH4s4s", version_ihl, 0, total_length, 0x1C46, 0x4000, 64, 6,
    0x087D, SRC_IP, DST_IP)
eth = struct.pack("!6s6sH", DST_MAC, SRC_MAC, 0x0800)
frame = eth + ip + tcp + data
```



- Each header is one `struct.pack`, and one wrong letter in its **format string** moves every later byte.
- A constant such as TTL 64 is **typed into every build**, header by header.
- The **total length** needs the inner bytes first, and both **checksums** are copied in, not computed.

One Expression: The Same 59 Bytes

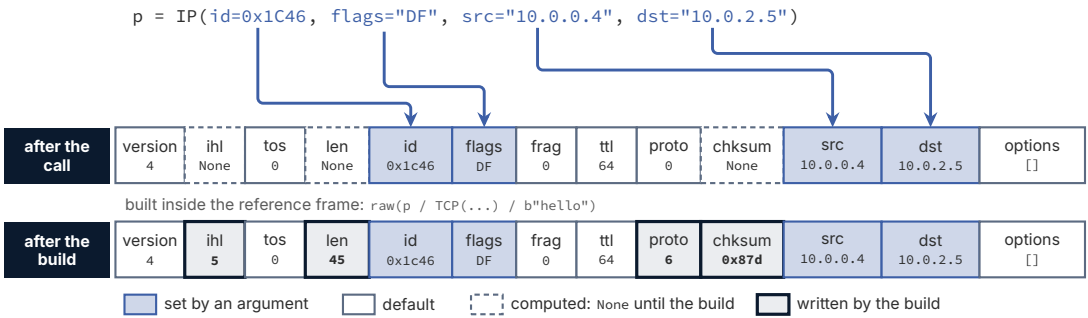
Scapy: one object per layer

```
from scapy.all import Ether, IP, TCP, raw
pkt = (Ether(dst="08:00:27:ab:cd:ef", src="08:00:27:12:34:56")
  / IP(src="10.0.0.4", dst="10.0.2.5", id=0x1C46, flags="DF")
  / TCP(sport=50000, dport=80, seq=SEQ, ack=ACK, flags="PA", window=0xFFFF) / b"hello")
raw(pkt) == frame   # True
```



- One expression builds the same 59 bytes: one object per layer, each value set by keyword, in any order.
- The build computes what the hand version copied in: the total length 45 and both checksums.
- A **layer object** holds one header as named values. **Scapy** is the Python library built on layer objects.

The Layer Object: Fields, Defaults and Computed Fields



- A **field** is one named value of a header, set by a keyword argument: the object holds all 13 of IPv4's, in wire order.
- A **default** is the value a field takes when no argument sets it: version 4, TTL 64, and the ID 1 when none is given.
- A **computed field** stays `None` until the packet is built, and the build fills it from the rest: IHL 5, total length 45, checksum `0x87d`. The build also rewrites the protocol from its default 0 to 6, for the TCP layer stacked on it.

The Interactive Session: Build, Inspect, Change

python3, one line at a time

```
>>> from scapy.all import IP, ls
>>> p = IP(dst="10.0.2.5")          # build: a layer object with one field set
>>> p                               # inspect: the object prints itself
<IP  dst=10.0.2.5 |>                # only the fields that differ from their defaults

>>> p.ttl                           # read one field
64                                  # the default

>>> p.ttl = 10                      # change: assign to the field
>>> p
<IP  ttl=10 dst=10.0.2.5 |>         # the changed field now prints

>>> p.len is None
True                                # the length is not built yet
```

- An **interactive session** runs each line as it is typed, so a packet can be built, read and changed one step at a time. The `scapy` command opens one with every Scapy name already imported.
- A layer object prints only the fields that differ from their defaults, so the changed `ttl` appears in the next line it prints.

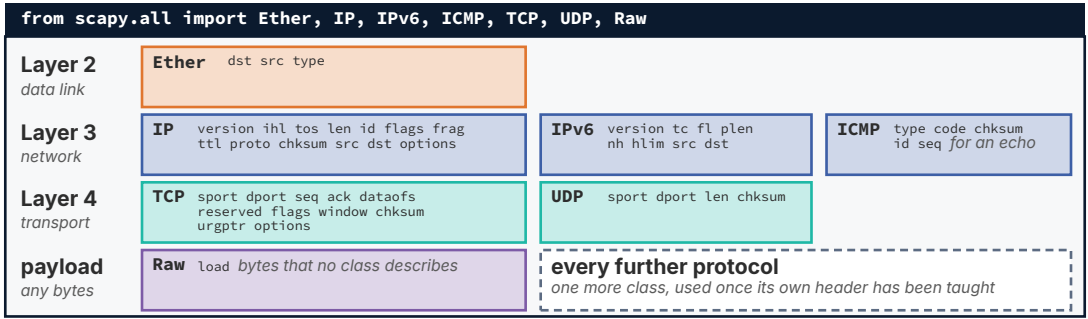
Listing a Class: Every Field, Its Type and Default

python3, the same session

```
>>> ls(IP)                                                  # every IPv4 field
version    : BitField  (4 bits)              = ('4')
ihl        : BitField  (4 bits)              = ('None')     # the build fills it
tos        : XByteField                      = ('0')
len        : ShortField                      = ('None')     # the build fills it
id         : ShortField                      = ('1')
flags      : FlagsField                      = ('<Flag 0 ()>')
frag       : BitField  (13 bits)             = ('0')
ttl        : ByteField                       = ('64')
proto      : ByteEnumField                   = ('0')
chksum     : XShortField                     = ('None')     # the build fills it
src        : SourceIPField                   = ('None')     # the build fills it
dst        : DestIPField                     = ('None')     # the build fills it
options    : PacketListField                 = ('[]')
```

- `ls(IP)` lists every IPv4 field with its type and default, and `None` marks a field the build fills in.
- The type sets what a field can hold: a `ByteField` is one byte, so TTL takes 0 to 255.

Protocol Classes: One Class per Header



- A **protocol class** is a Python class for one header, and each object made from it is one layer: `IP()` is one IPv4 header.
- A class lists its header's fields in wire order, under short names such as `chksum`, and `ICMP` sits with the network layer because its messages travel inside IPv4.
- **Security:** a tool that writes any header writes forged and malformed ones as easily, and the attacks later in this module are built that way.

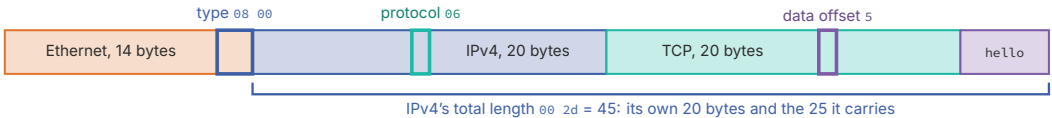
Crafting: Stacking Layers with /

Layer Binding: Each Header Names the Next

a reader starts at byte 0 and follows each header to the next



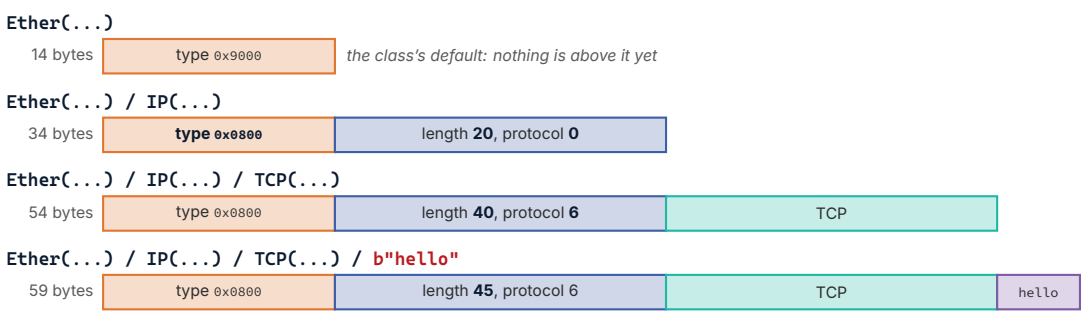
the same fields in the reference frame's 59 bytes



- Each lower header **names the next one**: Ethernet's type `0x0800` says IPv4 follows, and IPv4's protocol 6 says TCP follows.
- IPv4 also **counts what it carries**: its total length, 45, covers its own 20 bytes and the 25 inside it.
- **Binding** is Scapy's rule that writes these fields when one layer is stacked on another, so neither is typed by hand.

The / Operator: Each Layer Goes Under the Last

Each step adds one layer with `/`. `(...)` stands for the reference frame's arguments.



navy bold: a value the last `/` wrote.

- `A / B` puts B under the last layer of A, so a stack is written from the outside in.
- Each `/` rewrites the lower layers: Ethernet's type turns from the default `0x9000` into `0x0800`, and IPv4's length grows to 45.
- The stack is written from the outside in, yet every length still counts the bytes under it.

Bindings Across the Taught Headers: One Pattern

Lower layer	Stacked as	Field Scapy fills	Value	What the value says
Ethernet	`Ether() / IP()`	`type`	`0x0800`	IPv4 follows
	`Ether() / IPv6()`	`type`	`0x86dd`	IPv6 follows
IPv4	`IP() / TCP()`	`proto`	`6`	TCP follows
	`IP() / UDP()`	`proto`	`17`	UDP follows
	`IP() / ICMP()`	`proto`	`1`	ICMP follows
IPv6	`IPv6() / TCP()`	`nh`	`6`	TCP follows, as in IPv4
	`IPv6() / UDP()`	`nh`	`17`	UDP follows

- One rule covers every header taught so far: the lower layer's **next-protocol field** takes the number of the layer stacked on it.
- Only the field's name changes: `type` in Ethernet, `proto` in IPv4, and `nh`, the next header, in IPv6.
- The numbers are registered: EtherTypes by the IEEE, and protocol numbers by IANA, the one list IPv4 and IPv6 share.

An Explicit Value Overrides the Binding

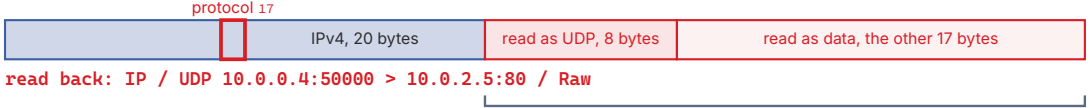
`(...)` stands for the reference frame's arguments.

```
IP(...) / TCP(...) / b"hello"     # protocol left unset: the binding writes 6
```



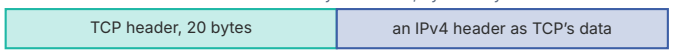

```
read back: IP / TCP 10.0.0.4:50000 > 10.0.2.5:http PA / Raw
```

```
IP(..., proto=17) / TCP(...) / b"hello"     # protocol set by hand
```




```
read back: IP / UDP 10.0.0.4:50000 > 10.0.2.5:80 / Raw
```

```
TCP() / IP()  # no order check
```




```
read back: TCP ftp_data > http S / Raw
```

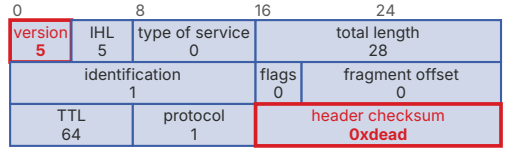
- A binding fills only a field left unset, so a protocol of 17 set by hand stays 17 above a TCP layer.
- Every reader follows that field: the same 25 bytes read back as an 8-byte UDP header and 17 bytes of data.
- Scapy checks no order either: `TCP() / IP()` builds 40 bytes, an IPv4 header carried as TCP's data.

Out-of-Specification Values: Limited Only by the Field Type

python3, one session

```
>>> from scapy.all import IP, ICMP, raw
>>> bad = IP(version=5, chksum=0xDEAD)  # no error
>>> raw(bad / ICMP())[:12].hex(" ")     # builds
'55 00 00 1c 00 01 00 00 40 01 de ad'

>>> p = IP(ttl=300)                     # no error
>>> p.ttl                               # stored
300
```



The 12 bytes printed at left, field by field. Version 5 fits the 4-bit field (0 to 15), and `0xdead` fits the checksum's 16 bits.

```
>>> raw(p)                              # the build packs every field
[29 lines: two chained tracebacks, 9 frames]
ValueError: While building field 'ttl': Incorrect type of value for field ttl:
struct.error(''B' format requires 0 <= number <= 255')
To inject bytes into the field regardless of the type, use RawVal. See help(RawVal)
```

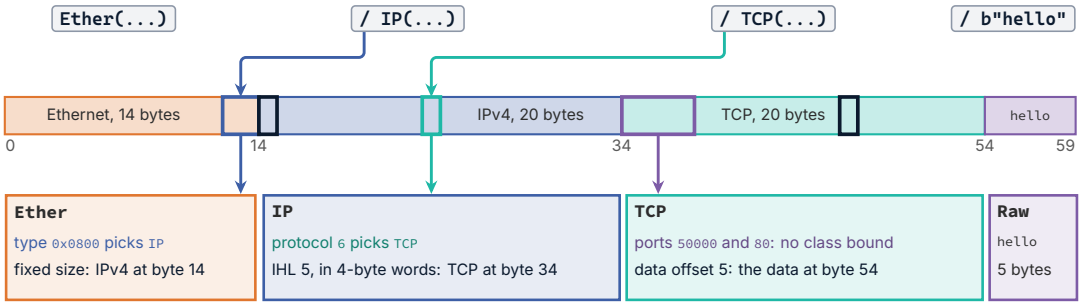
refused at the build: TTL is one byte (0 to 255), and 300 cannot be packed into it

- A field takes **any value its type can encode**: version 5 and checksum `0xDEAD` both build.
- A value the type cannot encode is refused at the build, not when it is assigned.
- **Security:** malformed and evasive packets are built this way, and later attacks start here.

Dissecting: From Bytes to Fields

Dissection: Bindings Read in Reverse

crafting writes each field from the layer stacked on it, and dissection reads it back to pick that layer's class



- Received bytes carry no layer objects: each header must tell a reader which protocol follows and where.
- The next-protocol field answers which, and a length answers where: Ethernet is always 14 bytes, and IPv4's IHL and TCP's data offset count 4-byte words.
- **Dissection** runs the bindings in reverse, from the outermost header in: each field picks the next class, and the bytes that no binding claims stay `Raw`.

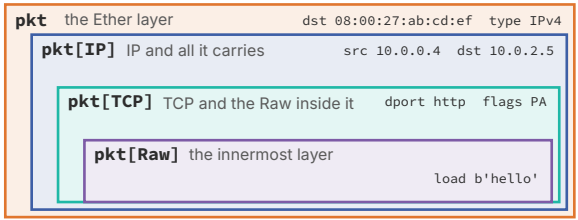
Reading a Packet: `.summary()` and `.show()`

python3, one session

```
[5 lines: the import, SEQ and ACK, and pkt built from the reference frame's arguments]
>>> pkt.summary()                                          # one line for the whole packet
'Ether / IP / TCP 10.0.0.4:50000 > 10.0.2.5:http PA / Raw'
>>> pkt.show()                                             # 33 lines, set in three columns
###[ Ethernet ]###                ###[ IP ]###                          ###[ TCP ]###
  dst       = 08:00:27:ab:cd:ef       version   = 4                           sport     = 50000
  src       = 08:00:27:12:34:56       ihl       = None                        dport     = http
  type      = IPv4                    tos       = 0x0                         seq       = 1715004
                                      len       = None                        ack       = 15070887
# one block per layer, each          id        = 7238                        dataofs   = None
# indented one step deeper           flags     = DF                          reserved  = 0
# than the layer carrying it         frag      = 0                           flags     = PA
                                      ttl       = 64                          window    = 65535
# IPv4, tcp, http: numbers           proto     = tcp                         chksum    = None
# printed by their names             chksum    = None                        urgptr    = 0
                                      src       = 10.0.0.4                    options   = []
# None: not built yet                dst       = 10.0.2.5                ###[ Raw ]###
                                      \options   \                              load      = b'hello'
```

- `.summary()` returns one line per packet: its layers, its addresses and ports, and TCP's flags.
- `.show()` prints every field of every layer, as the object holds them now, before any build.

Indexing: Reaching Any Layer or Field

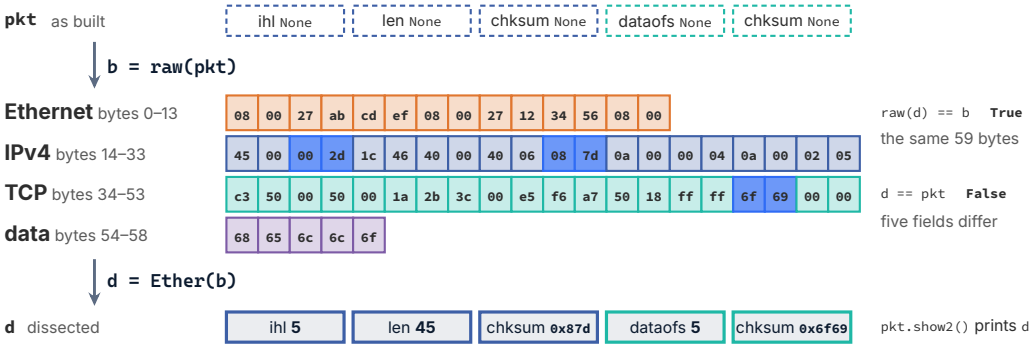


python3, the same session

```
>>> pkt[IP].src
'10.0.0.4'
>>> pkt[TCP].flags
<Flag 24 (PA)>
>>> pkt[Raw].load
b'hello'
>>> TCP in pkt
True
>>> UDP in pkt
False
>>> pkt[UDP]
[6 lines: the traceback]
IndexError: Layer [UDP] not found
```

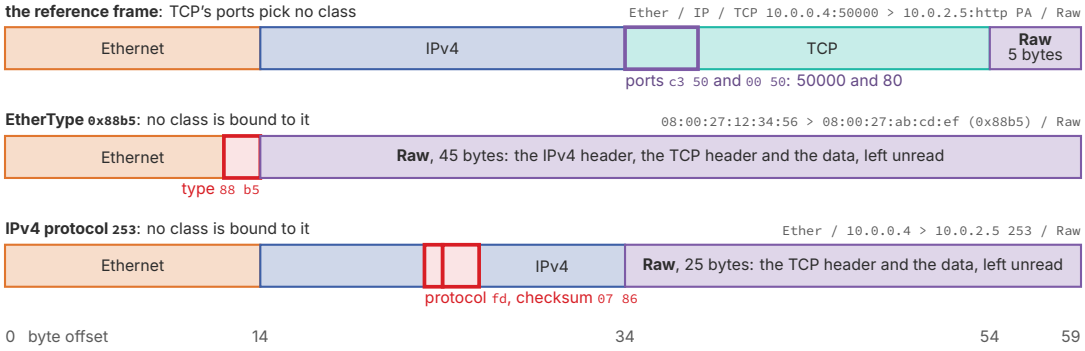
- **Indexing** by a class, as in `pkt[TCP]`, returns that layer, and a layer carries every layer inside it, so `pkt[TCP]` holds the `Raw` bytes too.
- One more dot reads a field: `pkt[IP].src`. TCP's flags print as their number and their letters, 24 and PA.
- `UDP in pkt` tests for a layer first, because indexing a missing one raises `IndexError`.

The Round Trip: Object to Bytes and Back



- The **round trip** builds the bytes with `raw(pkt)` and dissects them back with `Ether(b)`. `hexdump(pkt)` prints the same bytes, 16 to a row.
- The bytes survive it exactly, and the object does not: `d` holds every field set, while `pkt` left five to the build, so the two compare unequal.
- `show2()` is `show()` of this round trip, so it prints each computed field as it will be sent.

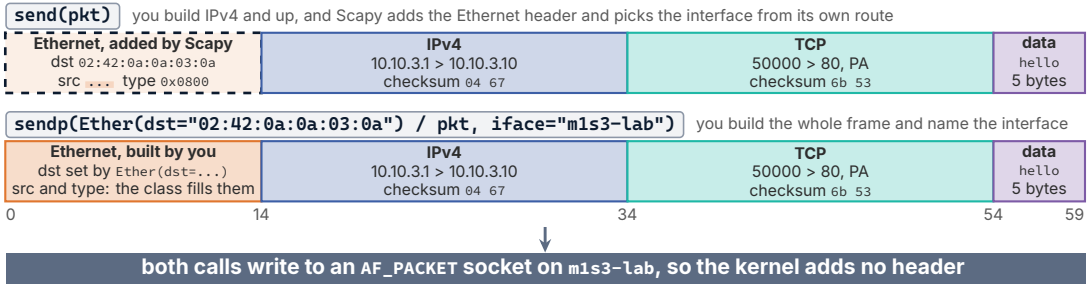
Unrecognized Bytes: The `Raw` Layer



- Where no binding matches, dissection stops, and Scapy keeps the rest of that header's payload as one `Raw` layer: five bytes in the reference frame.
- EtherType `0x88b5` and IP protocol 253 are both reserved for experiments, so Scapy binds no class to either.
- **Security:** an unfamiliar protocol arrives as `Raw`, and reading those bytes field by field is the first step in attacking it.

Sending: Packets onto the Wire

`send` and `sendp`: Which Call Adds the Ethernet Header



captured on the bridge: the two frames are byte-identical, 59 bytes, and `...` is the bridge's own MAC, new at each `compose up`

- A packet reaches the link only inside a frame: something must pick the interface it leaves by and write the Ethernet header in front of it.
- `send()` takes the packet from IPv4 up and does both itself. `sendp()` takes the frame from Ethernet up, sends it on the interface named, and adds no layer.
- So `send` is Scapy's **layer-3 send** and `sendp` its **layer-2 send**: the same bytes reach the wire either way, and only the author of the header differs.

Before `send`: Scapy's Own Route and MAC Lookup

1 Route lookup
in `conf.route`: the interface, source and gateway

2 MAC lookup
`getmacbyip()`: the MAC of the next hop

3 Ethernet header written
dst from step 2, src from the interface

Example 1: on the link, the target 10.10.3.10

```
pkt.route()
('m1s3-lab', '10.10.3.1', '0.0.0.0')
```

→

```
getmacbyip("10.10.3.10")
'02:42:0a:0a:03:0a'
```

→

dst 02:42:0a:0a:03:0a

sent out of `m1s3-lab`

Example 2: off the link, 192.0.2.1

```
conf.route.route("192.0.2.1")
('eth0', '10.0.2.15', '10.0.2.2')
```

→

```
getmacbyip("192.0.2.1")
'52:54:00:12:35:02'
```

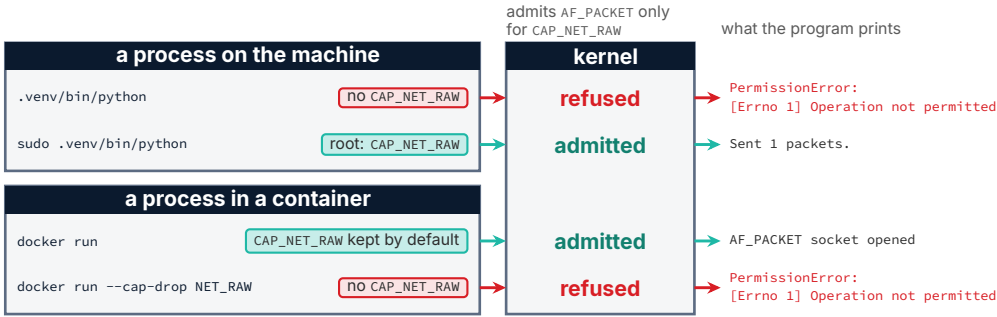
→

dst 52:54:00:12:35:02

sent out of `eth0`

- Before `send()` writes a frame, Scapy runs two lookups of its own: the route in **Scapy's routing table**, `conf.route`, then the MAC of the next hop.
- `conf.route` is a copy of the kernel's table, read when Scapy starts: a route added later is used only after `conf.route.resync()`.
- `getmacbyip()` asks for the next hop's MAC with a request Scapy sends itself, not the kernel's, and keeps each answer for 120 seconds.

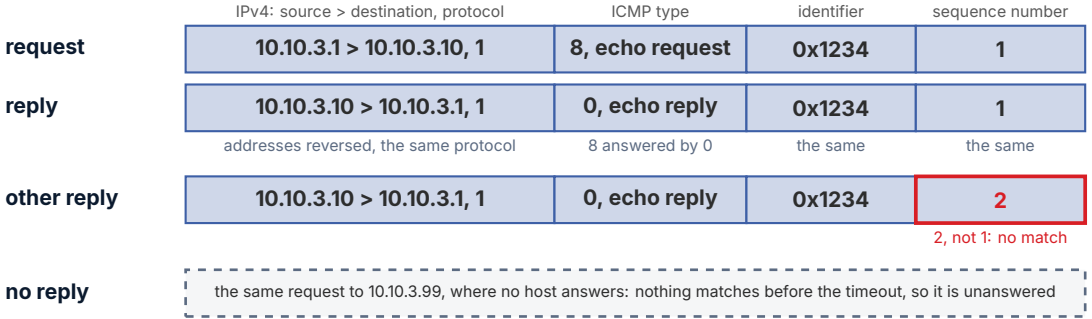
The Privilege Boundary: Every Send Needs `CAP_NET_RAW`



- Every Scapy send opens an `AF_PACKET` socket, and the kernel opens one only for a process that holds `CAP_NET_RAW`: without it, `send` and `sendp` raise `PermissionError`.
- Under `sudo` the interpreter runs as root, which holds it. A container's process holds it too, because Docker keeps `NET_RAW` by default, until `--cap-drop NET_RAW` removes it.
- **Security:** a library that hides the socket does not hide the privilege, and a container left with `NET_RAW` can forge frames on its network.

Send and Receive: `sr`, `sr1`, `srp`

Request and Reply: Matching an Answer to Its Probe



- A probe learns something only from its reply: the tool must catch it, tell it from the other packets on the link, and notice when none comes.
- A reply is matched by its fields: its addresses are the request's reversed, and an echo reply repeats the request's identifier and sequence number.
- A **timeout** ends the wait, and a request still unmatched is **unanswered**. Sending, waiting and matching together is **send-and-receive**.

`sr` and `sr1`: Answered and Unanswered



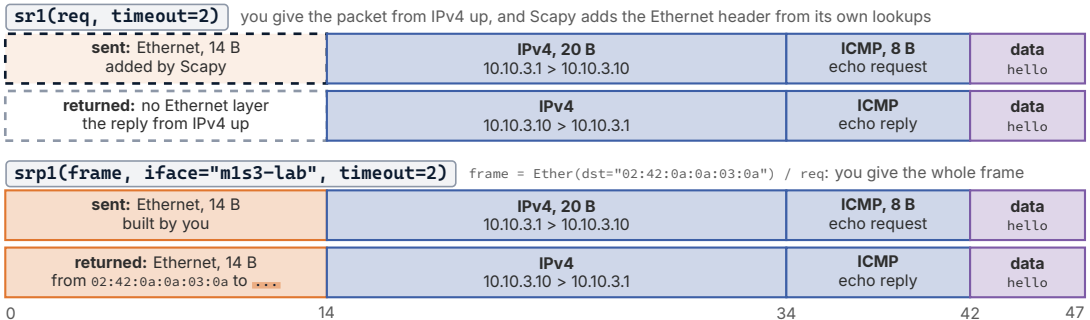
python3, one session, under `sudo`

```
[2 lines: the import, and req, the echo request to 10.10.3.10]
>>> ans, unans = sr(req, timeout=2)                  # send, wait up to 2 s, match
Begin emission
.*                                                   # *: an answer, .: a packet matching none
Finished sending 1 packets

Received 2 packets, got 1 answers, remaining 0 packets  # the extra one: the MAC lookup's answer
>>> conf.verb = 0                                    # return values only from here
>>> sr1(req, timeout=2).summary()                    # sr1: the first answer itself
'IP / ICMP 10.10.3.10 > 10.10.3.1 echo-reply 0 / Raw'
>>> req.dst = "10.10.3.99"                           # an address no host holds
>>> sr(req, timeout=2)                               # no MAC found, so sent to broadcast
WARNING: MAC address to reach destination not found. Using broadcast.
(<Results: TCP:0 UDP:0 ICMP:0 Other:0>, <Unanswered: TCP:0 UDP:0 ICMP:1 Other:0>)
```

- `sr()` keeps each received packet that matches a request, and returns the **answered pairs** and the **unanswered requests**.
- `sr1()` runs the same exchange and returns only the **first answer**, or `None` when nothing answered in time.

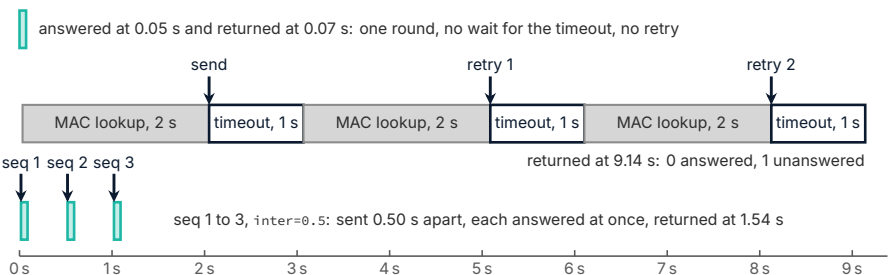
`srp` and `srp1`: The Same Exchange at Layer 2



- `srp()` and `srp1()` are the same exchange at **layer 2**: you supply the frame from Ethernet up and name the interface, as with `sendp`.
- At layer 2 the match adds one test before the IPv4 and ICMP ones: the reply's EtherType must be the request's.
- Use `srp` when the Ethernet fields are part of the question, such as a chosen destination MAC: `sr` always takes it from its own lookup.

Timeouts and Retries: When No Reply Comes

`sr(req, timeout=1, retry=2)` examples 1 and 2: the same call to two addresses, timed on the bridge from its start



- `timeout` is how long `sr` waits after its last send, in seconds. Left unset, an unanswered probe waits until it is interrupted.
- `retry=2` sends each unanswered request twice more. To an address no host holds, every round also pays Scapy's 2 s MAC lookup.
- `inter` spaces the sends of a list, in seconds. An empty answer list is a result too: nothing answered before the timeout.

One `sr1` Exchange: The Call, the Bytes and the Reply

the call, under `sudo`

```
[4 lines: the import, conf.verb = 0, req]
>>> rep = sr1(req, timeout=2)
>>> rep[ICMP].type, rep[ICMP].seq
(0, 1)
>>> hex(rep[ICMP].id)
'0x1234'
>>> rep[Raw].load
b'hello'
>>> rep[IP].src, rep[IP].dst
('10.10.3.10', '10.10.3.1')
>>> rep.answers(req)
1
```

the bytes sent

Ethernet	02 42 0a 0a 03 0a ... 08 00
IPv4	45 00 00 21 00 01 00 00 40 01 60 bd 0a 0a 03 01 0a 0a 03 0a
ICMP	**08** 00 a1 f8 **12 34 00 01**
data	**68 65 6c 6c 6f** *hello*

Scapy added the Ethernet header (dashed). `raw(req)` is the 33 bytes from IPv4 up, with ID `00 01` and checksum `60 bd`

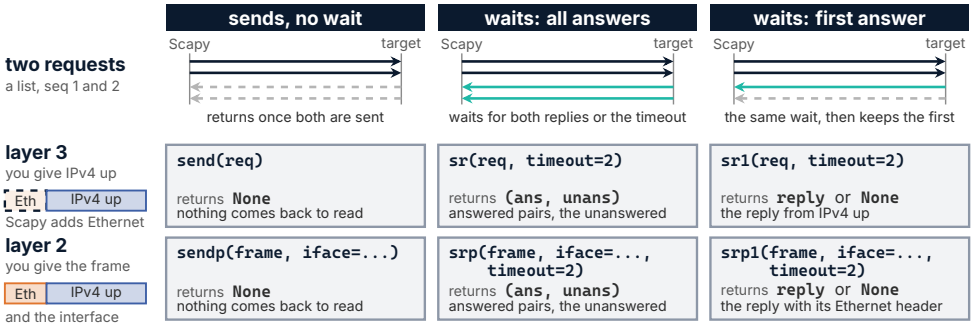
the reply

Ethernet	... 02 42 0a 0a 03 0a 08 00
IPv4	45 00 00 21 ... 00 00 40 01 ... 0a 0a 03 0a 0a 0a 03 01
ICMP	**00** 00 a9 f8 **12 34 00 01**
data	**68 65 6c 6c 6f** *hello*

The addresses come back reversed. The ID and checksum (...) are the target's, new on every reply

- One `sr1()` call is the whole loop: Scapy builds the 33-byte packet, sends it in a frame, and dissects the reply that matches into `rep`.
- Every field of the reply is then one index away, and `rep.answers(req)` runs the match itself: 1, an answer to this request.
- **Security:** this send, match and read loop is the core of scanning and probing: a later tool changes only the request it sends.

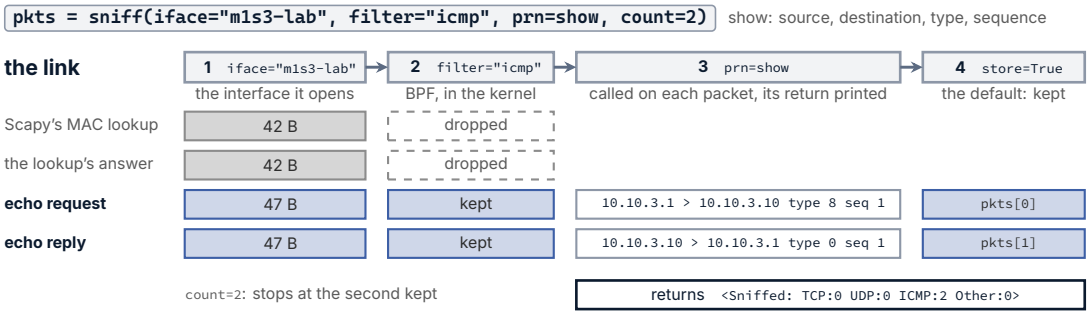
The Six Send Calls: Layer, Waiting and Return Value



- The row is the layer you supply: `send`, `sr` and `sr1` take the packet from IPv4 up, and `sendp`, `srp` and `srp1` the whole frame.
- Every call takes one packet or a list. Only the `sr` family waits, until every request is answered or the `timeout` ends.
- `sr1()` is `sr()` keeping only the first answer: given two requests, it waited for both and returned the reply to sequence 1.

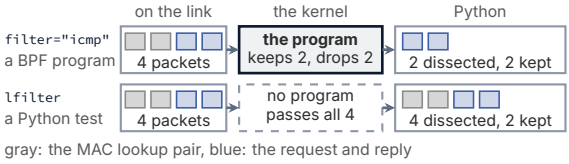
Capturing: `sniff()` and pcap

Capture: Interface, Filter, Action and Storage



- Watching traffic needs four choices: the interface to listen on, the packets to keep, what to do with each one, and whether to store it.
- `iface`, `filter`, `prn` and `store` make those choices. One `sr1` put four packets on the lab link, and the filter kept the request and its reply.
- Reading the packets that cross an interface, including ones you did not send, is **capture**: `sniff()` returns them as the layer objects you already build.

`sniff()`: Live Traffic as Packet Objects



the BPF program `filter="icmp"` compiles to

```
(000) ldh      [12]    # EtherType
(001) jeq      #0x800           jt 2    jf 5
(002) ldb      [23]    # IPv4 protocol
(003) jeq      #0x1             jt 4    jf 5
(004) ret      #65535  # keep the packet
(005) ret      #0      # drop it
```

terminal 1, under `sudo`

```
[1 line: the import]
>>> pkts = sniff(iface="m1s3-lab",
...     filter="icmp", count=2)  # blocks for 2
>>> pkts
<Sniffed: TCP:0 UDP:0 ICMP:2 Other:0>
>>> [p[ICMP].type for p in pkts]
[8, 0]                          # request, reply
>>> pkts[1][IP].src, pkts[1][ICMP].seq
('10.10.3.10', 1)               # the reply
```

terminal 2, under `sudo`

```
[2 lines: the import, conf.verb = 0]
>>> req = (IP(dst="10.10.3.10")
...    / ICMP(id=0x1234, seq=1) / b"hello")
>>> rep = sr1(req, timeout=2)  # the sender
>>> rep[ICMP].type, rep[ICMP].seq
(0, 1)
```

- `sniff()` blocks until it holds `count` packets or its `timeout` ends, then returns them as a list of layer objects, indexed as before.
- A `filter` is **BPF**: libpcap compiles it, and the kernel runs it before Python sees a packet. An `lfilter` is a Python test on every packet, after it is dissected.

`prn` and `AsyncSniffer`: Acting on Each Packet

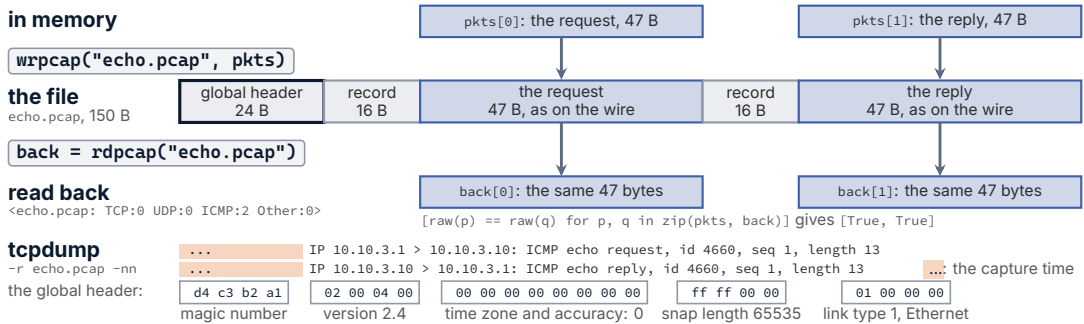


python3, one session, under `sudo`

```
[3 lines: the import, conf.verb = 0, and req, the echo request to 10.10.3.10]
>>> show = lambda p: f"{p[IP].src} > {p[IP].dst} type {p[ICMP].type} seq {p[ICMP].seq}"
>>> t = AsyncSniffer(iface="m1s3-lab", filter="icmp", prn=show)  # the arguments of sniff()
>>> t.start()                                                    # capture starts in a thread
>>> rep = sr1(req, timeout=2)                                    # the program sends meanwhile
10.10.3.1 > 10.10.3.10 type 8 seq 1                              # show's return, printed
10.10.3.10 > 10.10.3.1 type 0 seq 1
>>> pkts = t.stop()                                              # ends it, returns the list
>>> pkts
<Sniffed: TCP:0 UDP:0 ICMP:2 Other:0>
```

- `prn` is called on each packet as it arrives, and its return value is printed. With `store=False`, nothing is kept and the list returned is empty.
- `AsyncSniffer` runs the same capture in a thread: start it, send, then `stop()` returns the list. It takes no `timeout`: `join(timeout=...)` waits instead.
- `start()` returns before the socket is open. In a script, wait for `started_callback` before sending: without it, 7 of 10 runs caught nothing.

pcap Files: The Same Bytes Saved and Read Back



- A **pcap file** stores a capture: a 24-byte header naming the link type, then each frame behind a 16-byte record of its time and length.
- `wrpcap` writes a packet list to one, and `rdpcap` reads it back as a list: the same bytes, and the time to the microsecond.
- The format is shared: tcpdump reads the file Scapy wrote, so a capture made with one tool is read with another.

The Boundary: Three Tools for One Goal

Tool	Opens the capture	The filter	What you get	Taught
Scapy's `sniff()`	an `AF_PACKET` socket, opened by Scapy	BPF compiled by libpcap, or a Python `lfilter`	layer objects in a list, and pcap files	**here**
An external capture tool: tcpdump, Wireshark	libpcap, in a program of its own	BPF in the same syntax, compiled by libpcap	printed lines, pcap files, and pcapng from Wireshark	a later session
A sniffer built by hand	your own `AF_PACKET` socket and `recv`	your own Python code	raw bytes, dissected with `struct`	a later session

- Capture is one goal with three tools: Scapy's `sniff()` here, then an external capture tool and a sniffer built by hand on a raw socket in later sessions.
- All three read the same frames from the interface. They differ in who writes the capture code, and in the form the packets reach you.
- **Security:** capture is the defender's half of every attack that follows: whatever an attack sends crosses a link, where a capture can record it.

Key Takeaways

- Scapy makes each header a **layer object**: fields with defaults, and computed fields such as lengths and checksums, filled when the packet is built.
- The `/` operator stacks layers, and each lower header's **binding** field names the one above. A value you set overrides it, and only the field type limits a value.
- **Dissection** runs the bindings in reverse: a next-protocol field picks the next class, a length places it, and bytes no binding matches stay `Raw`.
- `send` adds the Ethernet header from Scapy's own route and MAC lookup, `sendp` sends your frame on the interface you name, and both need `CAP_NET_RAW`.
- The `sr` family sends, waits and **matches** each reply to its request by its fields. Always set a `timeout`: an empty answer list is a result.
- `sniff()` returns live traffic as the same objects. Filter in the kernel with BPF, act on each packet with `prn`, and keep the capture as a pcap file.

How the Practical Guide Uses This

1. **Open the Practical Guide.** Work with Scapy against the lab: build and dissect packets, send them, probe the target and match its replies, and capture an exchange to a pcap file.
2. **Record as you go.** Code, outputs and screenshots go into the **Submission worksheet** while you work, not at the end.
3. **Then the Home Task.** Apply the same method on your own. It has its own worksheet.

Keep the **Tooling Reference** (Scapy's calls, their arguments and what they return) and the **Troubleshooting FAQ** beside you throughout.

Open the Practical Guide

`Module1_Session3_Practical_Guide.md`

Module 1 — Session 3
Scapy and Raw Packet Crafting